# LAB 3 REPORT

## 자바프로그래밍1

| | |
|---|---|
| 과목명 | 자바프로그래밍 1 |
| 지도교수님 | 박경신 교수님 |
| 학과/학번 | 컴퓨터공학과/32234387 |
| 이름 | 조혜준 |

### I. 리포트 개요 및 설명

Java에서 클래스를 이용하여 날씨에 관련된 데이터를 입력 받고 파싱, 파싱된 데이터를 이용하여 사용자가 검색하고 싶어하는 조건을 이용하여 데이터를 찾고 그 데이터들과 데이터들의 최고, 최저 기온의 막대 그래프를 출력한다.

## 1. WeatherData

```java
public class WeatherData {
    public static String[][] data = {
        // columns: cityName, countryName, latitude, longitude, forecastDate, forecastWeather, forecastMinTempC, forecastMaxTempC
        {"Santiago", "Chile", "-33.445", "-70.6825", "2026-01-28", "Sunny", "15", "34"},
        {"Tashkent", "Uzbekistan", "41.27", "69.27", "2026-01-28", "Snow", "0", "2"},
        {"Addis Ababa", "Ethiopia", "9.02", "38.73", "2026-01-28", "Sunny", "12", "24"},
        {"Athens", "Greece", "37.97", "23.72", "2026-01-28", "Fair", "7", "16"},
        {"Tokyo", "Japan", "35.68", "139.77", "2026-01-28", "Cloudy", "2", "9"},
        {"Stockholm", "Sweden", "59.33", "18.05", "2026-01-28", "Sunny Intervals", "-6", "-2"},
        {"Paris", "France", "48.821667", "2.337778", "2026-01-28", "Partly Cloudy", "6", "8"},
        {"Madrid", "Spain", "40.411111", "-3.678056", "2026-01-28", "Rain", "3", "11"},
        {"Rome (ROMA)", "Italy", "41.951944", "12.500833", "2026-01-28", "Thunderstorms", "8", "15"},
        {"Moscow", "Russian Federation", "55.833333", "37.616667", "2026-01-28", "Snow", "-8", "-7"},
        {"Riyadh", "Saudi Arabia", "24.72", "46.73", "2026-01-28", "Fine", "27", "39"},
        {"Abu Dhabi", "United Arab Emirates", "24.433333", "54.466667", "2026-01-28", "Fair", "14", "25"},
        {"New Delhi (SFD)", "India", "28.583333", "77.2", "2026-01-28", "Fog", "12", "18"},
        {"Seoul", "Republic of Korea", "37.57", "126.97", "2026-01-28", "Clear", "-10", "-2"},
        {"Bangkok", "Thailand", "13.706944", "100.568056", "2026-01-28", "Cool", "23", "34"},
        {"Singapore", "Singapore", "1.366667", "103.916667", "2026-01-28", "Showers", "24", "33"},
        {"Beijing", "China", "39.933333", "116.283333", "2026-01-28", "Cloudy", "-6", "1"},
        {"Nairobi", "Kenya", "-1.3", "36.75", "2026-01-28", "Sunny Intervals", "15", "28"},
        {"Ottawa, Ontario", "Canada", "45.3225", "-75.669167", "2026-01-28", "Sunny Periods", "-21", "-11"},
        {"Washington DC", "United States of America", "38.85", "-77.03", "2026-01-28", "Cloudy", "-13", "-5"},
        {"Havana", "Cuba", "23.17", "-82.35", "2026-01-28", "Isolated Showers", "13", "24"},
        {"Buenos Aires", "Argentina", "-34.58", "-58.48", "2026-01-28", "Mostly Cloudy", "24", "33"},
        {"Wellington", "New Zealand", "-41.32", "174.82", "2026-01-28", "Fine", "11", "22"},
        {"Canberra", "Australia", "-35.308889", "149.200278", "2026-01-28", "Partly Cloudy", "15", "42"},
        {"Jakarta", "Indonesia", "-6.17", "106.8", "2026-01-28", "Rain", "22", "28"},
        {"Abuja", "Nigeria", "9.066667", "7.483333", "2026-01-28", "Dust", "21", "37"},
        {"London", "United Kingdom of Great Britain and Northern Ireland", "51.504722", "-0.131111", "2026-01-28", "Cloudy", "5",
        "9"},
        {"Ankara", "Türkiye", "39.95", "32.88", "2026-01-28", "Rain", "3", "7"},
        {"Berlin", "Germany", "52.565556", "13.310833", "2026-01-28", "Cloudy", "-3", "1"},
        {"Suva", "Fiji", "-18.15", "178.45", "2026-01-28", "Showers", "22", "32"}
    };

    // Weather 객체 배열로 변환하여 반환하는 메서드
    public static String[][] WeatherDatas(){
        return data;
    }
}
```

과제에서 제시된 데이터를 따로 클래스에서 static 형태로 저장하여 객체를 만들지 않아도 이용할 수 있도록 하고, 또한 static 메서드를 만들어서 이 데이터들을 String의 리스트 형태로 return 하도록 한다.

## 2. WeatherParser

```java
public class WeatherParser {
    //문자열 데이터 배열로부터 Weather 객체 배열을 생성
    public static Weather[] parse(String[][] data) {
        Weather[] weathers = new Weather[data.length];
        for(int i = 0; i < data.length; i++) {
            weathers[i] = new Weather(data[i]);
        }
        return weathers;
    }

}
```

이 클래스 또한 static 메서드를 만들어서 객체를 만들지 않아도 이용할 수 있도록 한다. 이 클래스의 메서드를 호출할 때 2차원 배열의 String을 입력받고, 이 입력받은 데이터의 길이만큼 반복하여 Weather 객체의 배열에 Weather 객체로써 저장하고 그 배열을 return 한다.

## 3. Weather

도시와 나라 이름, 위도, 경도, 예보 날짜, 예보 날씨, 최고 기온, 최저 기온을 String의 배열 형태로 받아 저장하고, 각 원소들을 반환하는 getter와 저장된 원소들의 문자열 표현을 반환

하는 toString 메서드를 가지는 Weather 객체를 만드는 클래스이다.

```java
import java.time.LocalDate;

public class Weather {
    private String cityName;
    private String countryName;
    private double latitude;
    private double longtitude;
    private LocalDate forecastDate;
    private WeatherIcon forecastWeather;
    private int forcastMinTempC;
    private int forcastMaxTempC;

    // 문자열 배열을 받아 Weather 객체를 생성하는 생성자
    public Weather(String[] condition) {
        this.cityName = condition[0];
        this.countryName = condition[1];
        this.latitude = Double.parseDouble(condition[2]);
        this.longtitude = Double.parseDouble(condition[3]);
        this.forecastDate = LocalDate.parse(condition[4]);
        this.forecastWeather = WeatherIcon.fromString(condition[5]);
        this.forcastMinTempC = Integer.parseInt(condition[6]);
        this.forcastMaxTempC = Integer.parseInt(condition[7]);
    }
```

```java
    // 예보 날짜 반환
    public LocalDate getForecastDate() {
        return this.forecastDate;
    }

    // 예보 날씨 아이콘을 반환합니다.
    public WeatherIcon getForecastWeather() {
        return this.forecastWeather;
    }

    // 최저 기온을 반환
    public int getForcastMinTempC() {
        return this.forcastMinTempC;
    }

    // 최고 기온을 반환
    public int getForcastMaxTempC() {
        return this.forcastMaxTempC;
    }
```

```java
    // 도시 이름 반환
    public String getCityName() {
        return this.cityName;
    }

    // 국가 이름 반환
    public String getCountryName() {
        return this.countryName;
    }

    // 위도 반환
    public double getLatitude() {
        return this.latitude;
    }

    // 경도 반환
    public double getLongtitude() {
        return this.longtitude;
    }
```

```java
    // Weather 객체의 문자열 표현을 반환
    @Override
    public String toString() {
        return "|" +
            " cityName='" + getCityName() + "'" +
            ", countryName='" + getCountryName() + "'" +
            ", latitude='" + getLatitude() + "'" +
            ", longtitude='" + getLongtitude() + "'" +
            ", forecastDate='" + getForecastDate() + "'" +
            ", Weather='" + getForecastWeather() + "'" +
            ", MinTempC='" + getForcastMinTempC() + "'" +
            ", MaxTempC='" + getForcastMaxTempC() + "'" +
            "|";
    }
```

## 4. WeatherFinder

날씨 데이터를 찾는 클래스를 만든다.

```java
// 날씨 데이터를 파싱하는 클래스
public class WeatherFinder {
    private Weather[] data;

    public WeatherFinder(Weather[] data) {
        this.data = data;
    }
```

도시 이름을 이용하여 입력된 도시의 이름과 같은 원소가 있는지 찾고 찾은 도시의 개수만큼의 공간을 가지는 배열을 만든다.
그후 그 배열에 그 도시의 정보를 넣고 return 하는 메서드를 만든다.

```java
// 도시 이름으로 날씨 데이터를 검색
public Weather[] findCityName(String cityname) {
    // 일치하는 항목 수 계산
    int count = 0;
    for (var weather : data) {
        if(weather.getCityName().equalsIgnoreCase(cityname)) {
            count++;
        }
    }

    // 결과 배열 생성 및 채우기
    int index = 0;
    Weather[] result = new Weather[count];
    for(var weather : data) {
        if (weather.getCityName().equalsIgnoreCase(cityname)) {
            result[index++] = weather;
        }
    }

    return result;
}
```

국가 이름을 이용하여서 위의 메서드와 같은 작업을 하는 메서드를 만든다.

```java
// 국가 이름으로 날씨 데이터를 검색
public Weather[] findCountryName(String countryname) {
    // 일치하는 항목 수 계산
    int count = 0;
    for (var weather : data) {
        if(weather.getCountryName().equalsIgnoreCase(countryname)) {
            count++;
        }
    }

    // 결과 배열 생성 및 채우기
    int index = 0;
    Weather[] result = new Weather[count];
    for(var weather : data) {
        if (weather.getCountryName().equalsIgnoreCase(countryname)) {
            result[index++] = weather;
        }
    }

    return result;
}
```

```java
// 날씨 아이콘으로 날씨 데이터를 검색
public Weather[] findIconWeathers(WeatherIcon icon) {
    // 일치하는 항목 수 계산
    int count = 0;
    for (var weather : data) {
        if(weather.getForecastWeather() == icon) {
            count++;
        }
    }

    // 결과 배열 생성 및 채우기
    int index = 0;
    Weather[] result = new Weather[count];
    for(var weather : data) {
        if (weather.getForecastWeather() == icon) {
            result[index++] = weather;
        }
    }

    return result;
}
```

대륙의 이름을 입력받고, 위도와 경도에 따라 대륙을 판정하고, 그 판정된 대륙의 이름에 따라 입력받은 대륙의 이름과 같은지 확인 후

```java
// 대륙으로 날씨 데이터를 검색
public Weather[] findContinent(String continent) {
    // 대륙별 좌표 범위 설정
    double minLat, maxLat, minLon, maxLon;
    if (continent.equalsIgnoreCase(anotherString: "Asia")) {
        minLat = 10; maxLat = 81; minLon = 26; maxLon = 170;
    } else if (continent.equalsIgnoreCase(anotherString: "Europe")) {
        minLat = 35; maxLat = 71; minLon = -25; maxLon = 66;
    } else if (continent.equalsIgnoreCase(anotherString: "Africa")) {
        minLat = -35; maxLat = 37; minLon = -17; maxLon = 51;
    } else if (continent.equalsIgnoreCase(anotherString: "North America")) {
        minLat = 7; maxLat = 83; minLon = -168; maxLon = -52;
    } else if (continent.equalsIgnoreCase(anotherString: "South America")) {
        minLat = -56; maxLat = 12; minLon = -81; maxLon = -35;
    } else if (continent.equalsIgnoreCase(anotherString: "Oceania")) {
        minLat = -47; maxLat = -10; minLon = 113; maxLon = 153;
    } else {
        // Antarctica
        minLat = -90; maxLat = -63; minLon = -180; maxLon = 180;
    }
    // 범위 내 항목 수 계산
    int count = 0;
    for(var weather : data) {
        if(weather.getLatitude() >= minLat && weather.getLatitude() <= maxLat
            && weather.getLongtitude() >= minLon && weather.getLongtitude() <= maxLon) {
            count++;
        }
    }

    // 결과 배열 생성 및 채우기
    int index = 0;
    Weather[] result = new Weather[count];
    for(var weather : data) {
        if(weather.getLatitude() >= minLat && weather.getLatitude() <= maxLat
            && weather.getLongtitude() >= minLon && weather.getLongtitude() <= maxLon) {
            result[index++] = weather;
        }
    }

    return result;
}
```

같다면 위의 메서드와 같이 배열에 저장 후 return 하는 메서드를 만든다.

온도에 대한 조건을 입력받고, 최고 기온이 0도 이상인지, 0도 이하인지 검색하여 그 결과를 위의 메서드와 같이 배열에 저장 후 return 하는 메서드를 만든다. 만약 유효하지 않은 조건이라면 아무것도 들어있지 않는 배열을 출력한다.

```java
// 온도 조건으로 날씨 데이터를 검색
public Weather[] findTemperature(String condition) {
    if (condition.equals(anObject: "under 0")) {
        // 최고 기온이 0도 이하인 항목 수 계산
        int count = 0;
        for (var weather : data) {
            if(weather.getForcastMaxTempC() <= 0) {
                count++;
            }
        }

        // 결과 배열 생성 및 채우기
        int index = 0;
        Weather[] result = new Weather[count];
        for(var weather : data) {
            if (weather.getForcastMaxTempC() <= 0) {
                result[index++] = weather;
            }
        }
        return result;
    } else if (condition.equals(anObject: "over 0")) {
        // 최고 기온이 0도 이상인 항목 수 계산
        int count = 0;
        for (var weather : data) {
            if(weather.getForcastMaxTempC() >= 0) {
                count++;
            }
        }

        // 결과 배열 생성 및 채우기
        int index = 0;
        Weather[] result = new Weather[count];
        for(var weather : data) {
            if (weather.getForcastMaxTempC() >= 0) {
                result[index++] = weather;
            }
        }

        return result;
    } else {
        // 유효하지 않은 조건
        return new Weather[0];
    }
```

## 5. WeatherIcon

```java
public enum WeatherIcon {
    SUNNY, SNOW, FAIR, CLOUDY, SUNNY_INTERVALS, PARTLY_CLOUDY, RAIN, THUNDERSTORMS, FINE, FOG, CLEAR, COOL, SHOWERS, SUNNY_PERIODS,
    ISOLATED_SHOWERS, MOSTLY_CLOUDY, DUST;

    // 문자열을 WeatherIcon으로 변환하는 메서드
    public static WeatherIcon fromString(String icon) {
        String format = icon.toUpperCase().replace(target: " ", replacement: "_");
        return WeatherIcon.valueOf(format);
    }
}
```

데이터에 있는 모든 날씨를 열거형으로 저장하고, 문자열을 입력 받으면 문자열과 같은 열거형 내의 원소를 반환하는 메서드를 가지는 열거형을 만든다.

## 6. WeatherGraph (Your Code)

```java
public class WeatherGraph {
    // 입력받은 데이터를 그래프로 출력하는 메서드
    public static void printGraph(Weather[] data) {
        if(data.length == 0) {
            System.out.println(x: "No data to display.");
            return;
        }

        System.out.println();
        System.out.println(x: "==================================================");
        System.out.println(x: "              최저/최고 기온 막대 차트              ");
        System.out.println(x: "==================================================");
```

```java
// 최저 기온 그래프
// 도시 이름을 먼저 출력하고, 0도를 기준으로 그래프를 그리는 좌측 여백을 만듬
System.out.printf(format: "%-20s | ", weather.getCityName());
// 0도 기준선 왼쪽 영역: 음수 온도가 있을 때는 ■로 표시
for (int i = -zeroOffset; i < 0; i++) {
    if (i < minTemp) {
        System.out.print(s: " ");
    } else {
        System.out.print(s: "■");
    }
}
// 0도 위치 표시
System.out.print(s: "|");
// 0도 기준선 오른쪽 영역: 양수 온도면 ■ 개수를 추가로 출력
for (int i = 0; i < Math.max(a: 0, minTemp); i++) {
    System.out.print(s: "■");
}
System.out.printf(format: " %d°C (Min)\n", minTemp);
```

```java
// 최고 기온 그래프
// 라벨을 비워서 같은 기준선을 아래에 동일하게 정렬
System.out.printf(format: "%-20s | ", ...args: "");
// 0도 기준선 왼쪽 영역: 최고 기온이 음수인 경우에 ■를 표시
for (int i = -zeroOffset; i < 0; i++) {
    if (i < maxTemp) {
        System.out.print(s: " ");
    } else {
        System.out.print(s: "■");
    }
}
// 0도 위치 표시
System.out.print(s: "|");
// 0도 기준선 오른쪽 영역: 양수 최고 기온을 시각화
for (int i = 0; i < Math.max(a: 0, maxTemp); i++) {
    System.out.print(s: "■");
}
System.out.printf(format: " %d°C (Max)\n", maxTemp);

System.out.println(x: "--------------------------------------------------");
```

입력받은 데이터에서, 최저 기온과 최고 기온을 추출하여 각 도시별 최저/최고 기온을 나타내는 막대그래프를 출력하는 static 메서드를 가지는 클래스를 만든다.

이때, 막대 그래프는 0의 위치를 기준으로 영하면 0의 왼쪽에, 영상이면 0의 오른쪽에 그래프를 그린다.

## 7. App (main)

```java
import java.util.Scanner;

public class App {
    public static Scanner scan = new Scanner(System.in);
    Run | Debug
    public static void main(String[] args) throws Exception {
        // 날씨 데이터를 파싱하여 Weather 객체 배열 생성
        Weather[] weathers = WeatherParser.parse(WeatherData.WeatherDatas());
        // WeatherFinder 객체 생성
        WeatherFinder finder = new WeatherFinder(weathers);

        // 사용자에게 검색 조건 입력 요청
        System.out.print(s: "Please enter the condition(cityName, countryName, weather, continent, temperature): ");
        String input = new String(scan.nextLine());
```

미리 scan을 설정하여 사용자에게 입력을 입력받을 수 있게 한다. 또한 날씨 데이터를 미리 파싱하여 Weather 객체 배열을 생성하고, 이를 이용한 WeatherFinder 객체를 생성한다.

```java
// 입력에 따라 검색 수행
switch(input){
    case "cityName":
        // 도시 이름으로 검색
        System.out.print(s: "Please enter the city name in english: ");
        String cityname = new String(scan.nextLine());

        Weather[] cityWeathers = finder.findCityName(cityname);

        System.out.println();
        System.out.println(x: " ------ Found Data by City ------");
        for (var weather : cityWeathers) {
            System.out.println(weather);
        }

        System.out.println();
        WeatherGraph.printGraph(cityWeathers);
        break;
```

```java
case "countryName":
    // 국가 이름으로 검색
    System.out.print(s: "Please enter the country name in english: ");
    String countryname = new String(scan.nextLine());

    Weather[] countryWeathers = finder.findCountryName(countryname);

    System.out.println();
    System.out.println(x: " ------ Found Data by Country ------");
    for (var weather : countryWeathers) {
        System.out.println(weather);
    }

    System.out.println();
    WeatherGraph.printGraph(countryWeathers);
    break;
```

```java
case "weather":
    // 날씨 아이콘으로 검색
    System.out.println(x: "Please enter weather in english");
    System.out.println(x: "(SUNNY, SNOW, FAIR, CLOUDY, SUNNY_INTERVALS, PARTLY_CLOUDY, RAIN, THUNDERSTORMS,
    System.out.print(s: ": ");
    WeatherIcon weathericon = WeatherIcon.fromString(scan.nextLine());

    Weather[] iconWeathers = finder.findIconWeathers(weathericon);

    System.out.println();
    System.out.println(x: " ------ Found Data by Weather ------");
    for (var weather : iconWeathers) {
        System.out.println(weather);
    }
```

```java
case "continent":
    // 대륙으로 검색
    System.out.print(s: "Please enter the continent name in english: ");
    String continent = new String(scan.nextLine());

    Weather[] continentWeathers = finder.findContinent(continent);

    System.out.println();
    System.out.println(x: " ------ Found Data by Continent ------");
    for (var weather : continentWeathers) {
        System.out.println(weather);
    }

    System.out.println();
    WeatherGraph.printGraph(continentWeathers);
    break;
```

```java
case "temperature":
    // 온도로 검색
    System.out.print(s: "Please enter which temperature you want to search(under 0, over 0): ");
    String temperature = new String(scan.nextLine());

    Weather[] temperatureWeathers = finder.findTemperature(temperature);

    System.out.println();
    System.out.println(x: " ------ Found Data by Temperature ------");
    for (var weather : temperatureWeathers) {
        System.out.println(weather);
    }

    System.out.println();
    WeatherGraph.printGraph(temperatureWeathers);
    break;
default:
    // 유효하지 않은 입력
    break;
```

입력받은 각 조건을에 대하여 WeatherFinder의 메서드를 이용하여 각 조건에 맞는 데이터를 찾아 Weather의 또 다른 배열에 저장하고, 이를 for문을 사용하여 출력 후, 그 배열을 이용하여 WeatherGraph의 printGraph 메서드를 이용하여 그래프를 출력한다.

입력받은 조건이 유효하지 않은 입력이라면 그대로 프로그램을 종료한다.